TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

Part 7

The branch tooling: objects to bones, the build sheet, the editor helper

Reference for the three things this branch ships: a Blender add-on that converts the TSC object hierarchy into a skinned skeletal mesh with value-scrubbed clips and sockets, the build sheet it writes for the numbers you type in the editor, and a Python script for the editor's own console that does the importing and the data assets. How each was tested, how to extend them, and what to script next.

In this part

1. Where the tool sits
2. Installing and using the add-on
3. What it reads: the tags
4. What it does, step by step
5. What it writes: the FBX set and the build sheet
6. The editor helper
7. How it was tested
8. Extending the tools
9. Further scripts worth writing
10. Known limits

Files: `tools/blender/tsw_export_prep.py`, `tools/tsw/tsw_editor_setup.py`, `tools/tsw/build_test.py`. The add-on was run headless in Blender 4.2's Python module on a synthetic locomotive and its output re-imported into Blender (**tested**). The editor helper was written against the UE 4.26 Python API and the field names read from the running editor but has not been executed inside it (**check**).

1 · Where the tool sits

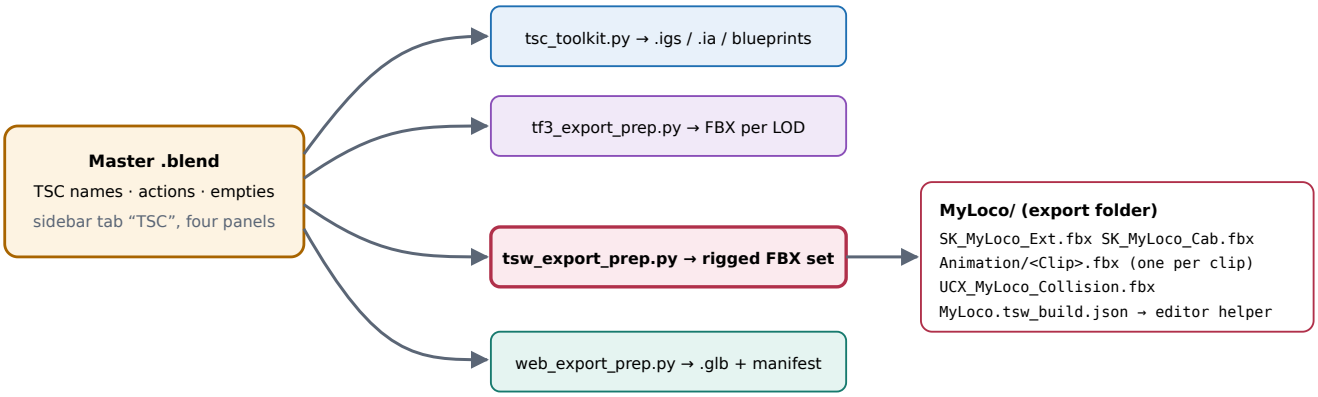


Figure 7.1 · Four panels, one tab, one master. The TSW panel is the only one that builds a rig; it does so on a temporary copy and deletes it after the export.

2 · Installing and using the add-on

1. Blender ▸ Edit ▸ Preferences ▸ Add-ons ▸ **Install…** ▸ `tools/blender/tsw_export_prep.py` ▸ tick it.
2. In the 3D viewport press `N` ▸ **TSC** tab ▸ panel **TSW branch**.
3. **Export root** (receives `<Name>/`), **Name** (`MyLoco` → `SK_MyLoco_Ext`), **Cab collection** (objects that become `SK_MyLoco_Cab`), wheel clip frames (60), bone length (cosmetic).
4. **Preview rig in scene**: builds the armature, the joined meshes, the sockets and the clip actions in a collection `_tsw_export`. Select the armature, open the Action editor, scrub a clip; check bones sit on hinges and sockets are where you expect. Delete the collection when done (or just export: the export rebuilds it).
5. **Export TSW FBX set + build sheet**. The report is in the Text Editor as `TSW_Export`: bones, sockets, clips, files.

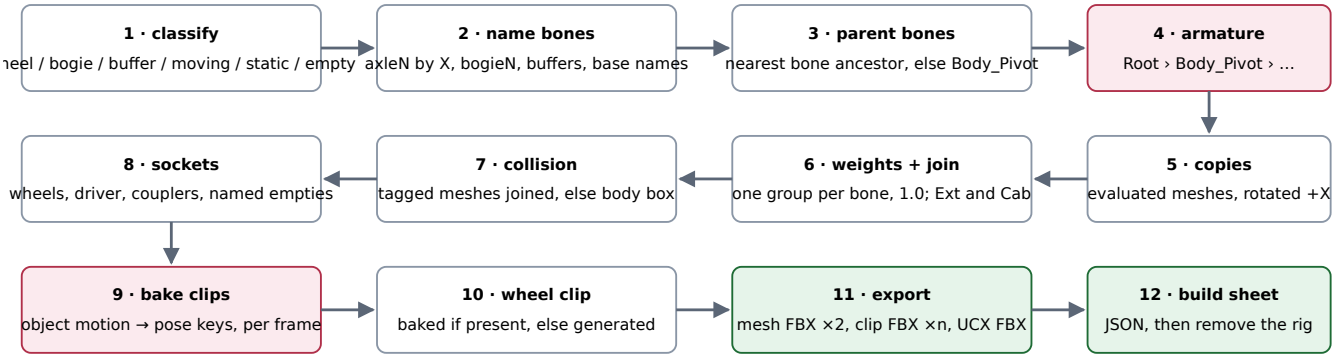
3 · What it reads: the tags

Property	On	Value	Effect
`tsw_bone`	Object	bone name	Force a bone for this object (even if static) and/or name it.
`tsw_clip`	Object	clip name	Name of the clip this object's animation goes into. Default: the NLA strip name, else `web_anim`, else the action name. Several objects with the same clip name bake into one clip.
`tsw_vhid`	Object	RVM component name	Recorded in the build sheet as “this VHID drives this clip” for the ABP's `VirtualHIDSequenceEntries`.
`tsw_socket`	Empty	socket name	Exported as `SOCKET_<name>` under the nearest bone ancestor. Empties named `socket_<name>` work without the property.

Property	On	Value	Effect
`tsw_collision`	Mesh	1	Joined into `UCX_<Name>_Collision` and exported as a static mesh instead of being skinned.
`tsw_cab`	Object	1	Goes into the cab mesh (same as being in the cab collection).
`tsw_skip`	Object	1	Left out.

Recognised without tags: wheel objects `…boNNwhNN` → `axleN`; bogie objects `…boNN` → `bogieN`; names containing “buffer” → `buffer_fl/fr/bl/br`; `tsc_driver` → `SOCKET_DriverSeat`; `tsc_coupling_front/rear` → `SOCKET_Coupler_F/B` + coupler numbers; any object with keys or strips → a bone. Dropped: LOD 2+, `shadow_`, `primarydigits`, `light_fwd/rev`. Keep the lower-case `axleN` / `bogieN` names: Thomas uses them and the editor's ABP auto-detects them (Part 2 §2).

4 · What it does, step by step



The master is never written to. The rig lives in a collection “_tsw_export” that the export deletes; the preview operator leaves it for you to inspect.

Figure 7.2 · The operator as a flow. Step 9 is the heart: for every frame it computes the object's motion since frame 0 through any moving parent and writes it as the bone's pose, so the skinned mesh reproduces the object animation exactly.

The FBX call for the mesh files, if you want to match it by hand:

```
bpy.ops.export_scene.fbx(filepath=path, use_selection=True, object_types={"ARMATURE", "MESH",
"EMPTY"},
    apply_unit_scale=True, apply_scale_options="FBX_SCALE_ALL", axis_forward="-Z", axis_up="Y",
    use_mesh_modifiers=True, mesh_smooth_type="FACE", add_leaf_bones=False,
use_armature_deform_only=False,
    primary_bone_axis="Y", secondary_bone_axis="X", bake_anim=False, path_mode="COPY")
# clips: object_types={"ARMATURE"}, bake_anim=True, scene range = clip range, scene renamed to
the clip (names the take)
```

5 · What it writes: the FBX set and the build sheet

File	Content
`SK_<Name>_Ext.fbx`	Armature + exterior mesh + all sockets. No animation.
`SK_<Name>_Cab.fbx`	Same armature + cab mesh (only if a cab collection / `tsw_cab` objects exist). Import onto the exterior's skeleton.
`Animation/<Clip>.fbx`	Armature only, one take named after the clip, frames 0..last. Includes `Wheels_FullRotation`.
`UCX_<Name>_Collision.fbx`	Static mesh for walkable collision.
`<Name>.tsw_build.json`	Below.

```
{
  "name": "DemoLoco",
  "units": "metres, Blender axes after rotation: X forward, Y left, Z up. UE: multiply by 100 for
cm, negate Y.",
  "physics": { "Width_m": 3.0, "Height_m": 4.16, "Length_m": 18.5,
               "PhysicsBodyOffset_xz_m": [0.0, 2.08], "CentreOfMassOffset_xz_m": [0.0, 1.66],
               "ApproximateBounds_cm": [1850, 300, 416], "Mass_kg": null },        // mass: from
the TSC blueprint
  "axles":  [ { "bone": "axle1", "x_m": 7.4, "radius_m": 0.525, "carrier": "bogie1",
"tsc_object": "1_0500_bo01wh01" }, … ],
  "bogies": [ { "bone": "bogie1", "x_m": 5.5 }, { "bone": "bogie2", "x_m": -5.5 } ],   //
SimuGraph Bogie node offsets
  "buffers_m": { "buffer_fl": [9.0, 0.9, 1.0], … },
  "couplers": [ { "side": "FRONT", "pivot_point_m": [9.5, 1.0], "socket": "Coupler_F" }, … ],
  "sockets": [ { "name": "Wheel_L1", "bone": "bogie1", "position_m": [7.4, 0.8, 0.0] }, … ],
  "clips":   [ { "name": "Door_L1", "frames": 24, "bones": ["door_l1"] }, … ],
  "vhids":   [ { "component": "CabDoor_L", "bone": "door_l1", "clips": ["Door_L1"] }, … ],  //
ABP VHID entries
  "bone_map": { "1_0500_door_l1": "door_l1", … },  "weights": { "1_0500_engine": "Body_Pivot", …
},
  "files": { … }
}
```

Every number the editor asks for in Part 4 §6, Part 5 §2 and Part 3 §8 is here, in metres with the Blender sign convention; the editor helper prints them converted.

6 · The editor helper

`tools/tsw/tsw_editor_setup.py` runs inside the editor (Output Log ▸ Cmd: `py "C:/path/tsw_editor_setup.py"`) after you edit its `CONFIG` block (sheet path, plugin root, friendly name, max speed). It then:

1. creates the plugin folder layout;
2. imports `SK_<Name>_Ext.fbx` as a skeletal mesh with a new skeleton, and the cab onto that skeleton;
3. imports every `Animation/*.fbx` as an AnimSequence on it;

4. imports the UCX static mesh (collision generation off);
5. creates `RVD_<Name>` and sets name, max speed, flags and bounds; links the RVM/RVV classes if they already exist;
6. creates an empty `FRM_<Name>`;
7. prints the physics numbers (Y negated), buffer and coupler offsets, the ABP list contents and the VHID entries.

It does not create the RVM, RVV or ABP Blueprints or the SimuGraph: component editing inside Blueprints is not exposed to Python in 4.26. Every step is wrapped so a failure prints and the script continues; property names that do not resolve are reported with "set it in the Details panel". check on first use: the data-asset factory call and the `RailVehicleDefinition` property names are the two things most likely to need a small edit.

7 · How it was tested

1. **Scene:** `tools/tsw/build_test.py` builds a TSC-named loco (+Y forward): body, roof, LOD-2 twin and a `shadow_` mesh (to be dropped), four doors keyed 0–24 with `tsw_vhid`, two bogies with three wheels each (cylinders rotated so their local axes do *not* match the world, to prove the world-aligned bones), a pantograph base → lower arm → upper arm → head chain keyed 0–30, a wiper with its action in an NLA strip, four buffers, a driver empty, two coupling empties, a custom audio socket, a cab collection with a throttle lever keyed 0–20, and a tagged collision mesh.
2. **Rig maths:** for the door, the pantograph (both arms, one as a child of the other), the wiper (NLA) and the lever, the bone's world pose at sampled frames matched the object's motion applied to the rest frame to within 1e-4; a vertex of the static pantograph head followed the upper-arm bone at frame 30; the generated wheel clip rolled the wheel top towards +X.
3. **Output:** 22 bones (Root, Body_Pivot, 4 doors, 2 bogies, 6 axles, 2 pantograph arms, wiper, 4 buffers, lever), 17 sockets, 8 clips (4 doors, pantograph, wipers, throttle, wheels), ext mesh 416 vertices, cab mesh 8 vertices, UCX, build sheet. No rig left in the scene afterwards.
4. **Round trip:** re-importing the exterior FBX gave the same bone tree with the same parents, every socket as an empty under the right bone at the right position, and the mesh at 18.5 × 3.0 × 4.16 m; importing `Door_L1.fbx` gave a take named `Door_L1` whose door bone sat at 79.8° on the last frame (keyed 80.2°); `Wheels_FullRotation.fbx` gave 174° at frame 29 of 60.

Repeat it with `python tools/tsw/build_test.py <out>` under the bpy wheel or `blender -b --python tools/tsw/build_test.py -- <out>`; it prints `ERRORS: []` when all checks pass.

8 · Extending the tools

Want	Change
A bone the game should move that has no animation yet (future sander, cab blind)	Give the object `tsw_bone`.

Want	Change
Rods and valve gear in the wheel clip	Key one revolution of wheels + rods in the master as an action named `Wheels_FullRotation` (or tag it `tsw_clip`); the add-on bakes it instead of generating the plain one.
Two doors on one button	Same `tsw_clip` on both; one clip, one VHID entry.
Hand-made LOD meshes	Export the LOD-2 twins as a second run with `tsw_skip` on LOD 1 and the twins renamed to LOD 1, then import as LOD levels. Or let the editor reduce.
Different forward rotation	`ROT_Y_TO_X` at the top of the file; identity if your master already faces +X.
Dense keys too heavy	`bake_anim_simplify_factor` in `export_tsw` (0 = keep every frame).
Materials from the TF3 branch	Export the PBR sheets with the TF3 packer, set roughness = 1 − gloss, repack ORM; the FBX only needs the base colour to find the material by name.

9 · Further scripts worth writing

Script	What it would save
VHID hitbox table	From the lever sockets, print each VirtualHIDInteractionComponent's attach socket, extent and interaction direction in UE centimetres so the RVV is filled from a list.
SimuGraph skeleton from the build sheet	The SA asset format is a Nodes[] array with typed structs; once the editor's Python can create one (or by reading a saved SA back), emit the Bogie and Axle nodes with offsets, gauge and radius from `axles` / `bogies`.
Control-value map from the TSC blueprint	Read the engine blueprint's control list (Regulator, Reverser…) and the .ia animation names and propose the `tsw_vhid` / `tsw_clip` tags, so a TSC-ready loco needs no manual tagging.
Thumbnail from Blender	Render the three-quarter view used by the web branch at TSW's thumbnail size as a stop-gap before the preview-scene plugin shot.
Batch export	Loop over all master .blend files headless (like `build_test.py` does) and refresh every vehicle's FBX set and build sheet in one run.
Post-import checker in the editor	Python in the editor: list the imported skeleton's bones and sockets and diff them against the build sheet; list the ABP's Axles/Bogies entries and warn about missing sim node references.

10 · Known limits

- Objects are converted, not armatures: if the master already has an armature (valve gear rigged with bones), its bones are not merged; export that mesh by hand.
- Shape keys are not baked into bones; a part that must deform (a bellows) needs a hand-made rig.
- Every bone is world-aligned (X forward, Y left, Z up). Fine for everything scrubbed from clips; if the engine's `Axles` spin axis turns out to be a bone-local axis other than the one the generated wheel clip uses, keep the clip route (exact) and tell the add-on's author.

- An object with two overlapping NLA strips in different tracks is baked as their blend, which is probably not what you keyed; keep one strip per object or set `tsw_clip` and separate the tracks.
- Materials and textures are not converted; the FBX carries names and base colour only. Build the master material in the editor.
- The editor helper is untested in the editor and creates no Blueprints; expect to fix one or two property names on first run.